

TESTES DE API COM POSTMAN Tutorial

completo para QAs — na prática com o ServeRest

Aula prática, do zero, para quem quer entender como testar uma API usando o Postman.

	Postman (Desktop ou versão Web/navegador)
	ServeRest — https://serverest.dev
	https://front.serverest.dev/login
	2 aulas de 50 minutos
	Nenhum conhecimento prévio de API é necessário

Ao final desta aula, o(a) aluno(a) será capaz de:

- Entender o que é uma API e como ela se relaciona com o front-end de um sistema.
- Relacionar os verbos HTTP (GET, POST, PUT/PATCH, DELETE) com as operações de CRUD.
- Instalar/acessar o Postman (aplicativo ou navegador) e criar uma Collection organizada em pastas.
- Consultar a documentação Swagger de uma API real (ServeRest) para descobrir endpoints.
- Cadastrar um usuário, autenticar-se (login) e usar o token recebido para autorizar novas requisições.
- Interpretar códigos de status HTTP (200, 201, 401, 405) a partir de erros reais encontrados na prática.
- Criar uma variável de ambiente (Environment) para reaproveitar a URL base em todas as requisições.

■ **Fala do professor:** Muita gente morre de medo de aprender teste de API achando que é complicado. Hoje vocês vão ver, na prática, que não existe mágica nenhuma nisso — é pura lógica. Peguem o computador de vocês e acompanhem passo a passo comigo, porque só assistir não faz vocês absorverem o conteúdo.

2.1 Front-end x Back-end (API)

O front-end é a parte do sistema programada para mostrar as coisas de forma bonita para o usuário. A API (back-end) é onde a lógica do sistema realmente acontece — é ela quem decide se uma ação pode ou não ser feita. No exemplo de hoje, vamos usar um e-commerce fictício (ServeRest) e comprovar que tudo o que aparece na tela também pode ser feito diretamente pela API.

2.2 CRUD e os verbos HTTP

Um sistema (e um banco de dados, no geral) trabalha em cima do conceito de CRUD:

C	Create	POST	Cria um novo registro
R	Read	GET	Lê/consulta um registro
U	Update	PUT / PATCH	Atualiza um registro existente
D	Delete	DELETE	Remove um registro

GET é para pegar informação, POST é para criar/enviar informação, PUT/PATCH é para atualizar e DELETE é autoexplicativo.

■ **Fala do professor:** O login, por exemplo, é sempre feito com POST, nunca com GET. Se fosse GET, o usuário e a senha poderiam acabar expostos na própria URL da requisição — e isso é um problema de segurança.

2.3 As ferramentas do dia

Postman é uma das ferramentas mais utilizadas do mercado para testar APIs, tanto por desenvolvedores quanto por QAs. Existem alternativas (como o Insomnia) que fazem praticamente a mesma coisa — a lógica aprendida aqui vale para qualquer uma delas. O Postman pode ser usado como aplicativo instalado (Windows, Mac ou Linux) ou direto pelo navegador, criando uma conta gratuita.

ServeRest é uma API pública e gratuita, em português, criada por Paulo Gonçalves para prática de testes. Ela simula um e-commerce completo, com usuários, produtos e login.

Download do Postman: <https://www.postman.com/downloads/>

Documentação (Swagger) do ServeRest: <https://serverest.dev/>

Front-end do e-commerce: <https://front.serverest.dev/login>

Antes de abrir o Postman, vale a pena mostrar o front-end do ServeRest na tela, para os alunos entenderem o que estão prestes a testar "por trás".

- Acessar <https://front.serverest.dev/login> e mostrar a tela de um usuário já logado.

• Observar a lista de produtos cadastrados (sem imagem — é assim mesmo por padrão do projeto).

- Abrir a documentação Swagger em <https://serverest.dev/> e mostrar como cada endpoint é documentado: o que ele espera receber e o que devolve.

- Destacar a Base URL do projeto, que aparece na própria documentação: **<https://serverest.dev>**.

■ **Fala do professor:** Reparem que o nome do site é literalmente igual à base URL documentada no Swagger: `serverest.dev`. Isso não é coincidência — é assim que toda API funciona: existe uma URL raiz, e cada endpoint é um 'caminho' a partir dela.

O

professor conduz cada passo no projetor, enquanto os alunos repetem no próprio computador.

No Postman, criar uma nova Collection em branco chamada **ServeRest Aula**. Escolher o tipo HTTP (não GraphQL nem gRPC).

Uma Collection é uma pasta que organiza um conjunto de requisições relacionadas ao mesmo projeto.

Criar uma requisição GET simples para <https://serverest.dev> (sem nenhum caminho extra) e clicar em Send.

✓ **Resultado:** Status 200 OK — a resposta devolve a própria página do Swagger em HTML. Isso confirma que a base URL está no ar e respondendo.

Criar uma nova requisição GET para <https://serverest.dev/produtos> e nomeá-la como **Listar produtos**. Organizar essa requisição dentro de uma pasta chamada **Produtos**.

✓ **Resultado:** Status 200 OK — a resposta traz a lista de produtos cadastrados no momento (quantidade pode variar, já que outras pessoas também testam essa API pública em tempo real).

■ **Fala do professor:** Se vocês atualizarem essa mesma listagem daqui a pouco, é bem provável que o número de produtos mude — porque essa é uma API pública, e outras pessoas estão testando ao mesmo tempo que a gente.

Criar uma nova requisição POST para <https://serverest.dev/usuarios>, organizada em uma pasta chamada **Usuários**, com o nome **Criar usuário**. No corpo (Body → raw → JSON), enviar:

```
{
  "nome": "Fulano da Silva",
  "email": "meuemail@qa.com.br",
  "password": "teste123",
  "administrador": "true"
}
```

É importante marcar **administrador: true**, porque só um usuário administrador consegue cadastrar produtos mais adiante.

✓ **Resultado:** Status 201 Created — status usado quando algo é criado com sucesso. A resposta traz uma mensagem de confirmação e o **id** do novo usuário.

Criar uma pasta **Login** com uma requisição POST para <https://serverest.dev/login>. No Body (raw/JSON), enviar o e-mail e a senha cadastrados no passo anterior:

```
{
  "email": "meuemail@qa.com.br",
  "password": "teste123"
}
```

✓ **Resultado:** Status 200 OK — a resposta traz a mensagem "Login realizado com sucesso" e um **authorization token** (um Bearer Token), que é a chave de acesso do usuário logado.

Esse token tem tempo de expiração e representa, nesse caso, o acesso de um administrador. Apenas fazer login não altera nada no sistema — ele só devolve essa chave.

Criar uma requisição POST para <https://serverest.dev/produtos>, na pasta **Produtos**, chamada **Cadastrar produto**, enviando no Body:

```
{
  "nome": "Notebook Positivo",
  "preco": 3200,
  "descricao": "Notebook, computador, laptop",
  "quantidade": 22
}
```

}

Enviar essa requisição **sem** configurar nenhuma autorização.

■ **Resultado:** Status 401 Unauthorized — o cadeado ao lado desse endpoint na documentação já avisava: essa ação exige autorização. Sem o token, o sistema recusa a criação do produto.

■ **Fala do professor:** Isso é ótimo pra mostrar pra vocês: mesmo tendo feito login, aquele token não vale de nada sozinho. A gente precisa efetivamente colocar ele na requisição.

Na requisição **Cadastrar produto**, ir até a aba **Authorization**, escolher o tipo **Bearer Token** e colar o token copiado da resposta do login (sem incluir a palavra "Bearer" antes — o Postman já adiciona isso automaticamente). Enviar novamente.

✓ **Resultado:** Status 201 Created — "Cadastro realizado com sucesso". Ao repetir o passo 3 (Listar produtos), o novo produto aparece na lista, comprovando que a API realmente gravou a informação.

Criar uma nova requisição DELETE, na pasta **Produtos**, chamada **Deletar produto**, apontando para `https://serverest.dev/produtos` (sem nenhum Body).

■ **Resultado:** Status 405 Method Not Allowed — a própria API explica: não é possível fazer DELETE direto em `/produtos`; é preciso indicar qual produto, informando o ID dele na URL.

Corrigir a URL para `https://serverest.dev/produtos/{id}`, usando o ID do produto criado no Passo 6/7 (copiado da resposta do cadastro), e configurar a mesma autorização Bearer Token.

✓ **Resultado:** Status 200 OK — "Registro excluído com sucesso". Ao listar os produtos novamente, o item não aparece mais. Se a mesma requisição for enviada de novo, a resposta passa a ser "Nenhum registro excluído", já que aquele ID não existe mais.

Para não precisar reescrever `https://serverest.dev` em toda requisição (e evitar retrabalho caso essa URL mude no futuro), criar um Environment:

- No canto superior direito do Postman, clicar em "Create Environment".
- Nomear o ambiente como **ServeRest Aula**.
- Criar uma variável chamada **base_url**, com o valor **https://serverest.dev** (sem barra no final).

Selecionar esse Environment no seletor de ambientes do Postman.

- Em cada requisição já criada, substituir a URL fixa por **{{base_url}}** seguido do caminho específico (ex.: `{{base_url}}/login`, `{{base_url}}/produtos`).
Ao digitar `{{` dentro da URL, o Postman já sugere as variáveis disponíveis — inclusive a `base_url` que acabamos de criar.

■ **Fala do professor:** Reparem que isso não muda o resultado dos testes — todos continuam funcionando igual. Mas agora, se a URL do projeto mudar um dia, a gente só precisa alterar em um único lugar: a variável de ambiente.

200 OK	GET na base URL / GET em /produtos / login bem-sucedido bem

201 Created	Cadastro de usuário e cadastro de produto bem-sucedidos
401 Unauthorized	Tentativa de cadastrar produto sem enviar o token de autorização
405 Method Not Allowed	Tentativa de DELETE em /produtos sem informar o ID do produto

Em duplas, repetir a sequência completa da aula, sem consultar as anotações sempre que possível, e depois resolver os desafios extras abaixo:

- Criar a Collection, as pastas (Produtos, Usuários, Login) e o Environment do zero. •

Cadastrar um novo usuário administrador com um e-mail diferente do usado em aula. •

Fazer login com esse novo usuário e obter o token.

- Cadastrar dois produtos diferentes, usando o Bearer Token.
- Listar os produtos e confirmar que ambos aparecem na resposta.
- Deletar apenas um dos dois produtos e comprovar, pela listagem, que só ele foi removido. • Repetir

a exclusão do mesmo produto uma segunda vez e registrar qual mensagem a API devolve.

■ **Fala do professor:** Se algo der erro, antes de me chamar, perguntem pra vocês mesmos: 'esse endpoint precisa de autorização?', 'meu body está no formato certo?', 'eu troquei a URL fixa pela variável em todo lugar?'. Grande parte dos erros de API é exatamente isso.

Organização da Collection			s, sem pastasRequisições pastas (Produtos, Usuários, organizada e nomeada de
Autenticação (login, token)			ter ou usar o token Obteve o ao menos uma requisição ken corretamente em todas
Leitura de erros (status code)			etar os erros 401/405 tros com apoio do professor viu os erros de forma
Uso de variáveis no ambiente			rel base_urlCriou a variável, todas as requisições Usou ma consistente em toda a

■ **Fala do professor:** Vocês viram que não existe mágica nenhuma nisso — é pura lógica. O Postman ainda tem muito mais recursos (documentação automática, IA integrada, coleções compartilhadas), mas o que vimos hoje já é a base de tudo. Pratiquem em casa, testem outros endpoints do próprio ServeRest e tragam suas dúvidas na próxima aula.